

UNIVERSIDAD AUTONOMA DE BAJA CALIFORNIA



FACULTAD DE CIENCIAS QUIMICAS E INGENIERIA

INGENIERIA EN COMPUTACION

**MATERIA:
SISTEMAS EMBEBIDOS**

**TEMA:
PRACTICA#3**

**ALUMNO:
LERMA GARCIA ERICK JOVAN – 01281695
GARCIA CHAVEZ ERICK - 01275863**

FECHER DE ENTREGA: 26 de SEPTIEMBRE del 2025

Objetivo

El alumno se familiarizará con el uso del periférico UART utilizando el sistema embebido ESP32 DevKit v1, para desarrollar aplicaciones en sistemas basados en microcontrolador, aplicándolos en la resolución de problemas de cómputo de manera eficiente y responsable.

Equipo

- Computadora personal con conexión a Internet.

Teoría

Describir el modo UART half-duplex del ESP32:

El ESP32 soporta comunicación UART en modo **half-duplex**, lo que significa que los datos pueden transmitirse y recibirse por un mismo canal, pero no de manera simultánea. En este modo, se utiliza un solo par de líneas para enviar y recibir información, alternando entre transmisión y recepción.

Los conceptos clave a considerar son:

- **UART (Universal Asynchronous Receiver/Transmitter):** Protocolo de comunicación serie asíncrona que permite la transmisión de datos entre dispositivos.
- **Half-duplex:** Solo un sentido de comunicación activo a la vez; el dispositivo debe cambiar entre transmitir y recibir.
- **Configuración de UART en ESP32:** Se puede establecer el baud rate, número de bits de datos, paridad, bits de parada y buffer de recepción/transmisión.
- **Uso de tareas y sincronización:** Para manejar la recepción y transmisión sin bloquear la ejecución del programa, se emplean tareas concurrentes y mecanismos como semáforos o colas en ESP-IDF.

El modo UART half-duplex es útil cuando se necesita comunicación bidireccional con ahorro de pines, como en proyectos con múltiples dispositivos conectados en serie.

Desarrollo

Implemente en dos ESP32, utilizando ESP-IDF, una pancarta ASCII haciendo uso de UART, tareas y mecanismos de sincronización de tareas. La implementación debe ser eficiente en el uso de recursos de cómputo (procesador, memoria y periféricos). La implementación se realizará en los boards ESP32, no en el simulador.

Un ESP32 recibe del usuario una cadena de texto de máximo 25 caracteres usando UART. Este ESP32 envía la cadena por UART al otro ESP32. El segundo ESP32 convierte los caracteres a un formato de pancarta ASCII y los despliega en la terminal mediante UART.

Limite los caracteres válidos a las letras del alfabeto en mayúscula y minúscula, números, espacio y los caracteres especiales !, ., + y -. La recepción de la cadena del usuario debe estar validada y debe aceptar las teclas Backspace y Enter.

La pancarta ASCII es de su elección, pero debe ser similar a las de los siguientes ejemplos y debe ser legible.

Haga uso de las funciones de **UART.h** para recibir, enviar y desplegar los caracteres. Es decir, utilice **uart_read_bytes**, **uart_write_bytes** y demás funciones de **UART.h**. No utilice **scanf**, **printf**, u otras funciones de entrada/salida estándar.

Realización de código ESP32-1

Para comenzar realiza la configuración correspondiente para el uso de uart en el esp32-1.

Utilizando la función `uart_read_bytes()` se espera a que el usuario ingrese una cadena de texto, una vez ingresada la cadena de texto esta la transmite con `uart_write_bytes` a el esp32-2.

Realización de código ESP32-2

Se realiza la configuración correspondiente para la recepción de datos y uso de UART en el esp32-2.

Utilizando la función `uart_read_bytes ()` espera para leer un dato mandado por el puerto de comunicación asignado para el esp32-1.

Una vez leído el dato se verifica que cumpla con los caracteres validos y sea una cadena válida para mostrar.

Una vez validado los datos, mediante código (ciclos for anidados y llamando a los datos de la pancarta) leemos la cadena recibida y así vamos desplegando los datos de la pancarta que se encuentran declarados en una constante al inicio del código.

Tanto el transmisor como el receptor marcan "LOGS" para el señalamiento de recepción y transmisión de datos.

Resultados:

Enviando un Hola en la transmisión por la terminal a esp32-1:

The screenshot shows the ESP-IDF command prompt interface. The terminal window displays the following commands and output:

```
a3\UART_USER\ESP_COMM-USER> & 'C:\Espressif\python_env\idf5.5_py3.11_env\Scripts\python.exe' 'C:\tools\idf_monitor.py'
-p COM4 -b 115200 --make "'C:\Espressif\Esspressif\frameworks\es
```

A context menu is visible over the terminal, listing navigation shortcuts:

- Navigate to Command
- Scroll to Previous Command (Ctrl+UpArrow)
- Scroll to Next Command (Ctrl+DownArrow)

The terminal output continues with the following messages:

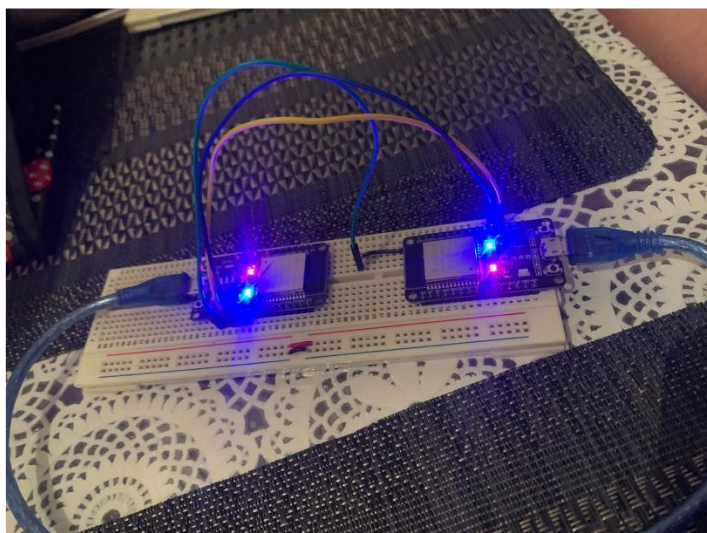
```
I (259) spi_flash: detected chip: generic
I (261) spi_flash: flash io: dio
W (264) spi_flash: Detected size(4096k) larger than the size in the binary image header(2048k). Using the size in the binary image header.
I (277) main_task: Started on CPU0
I (287) mI (287) ESP_EMISOR: UARTs configurados. UART_ESP_COMM en TX:4, RX:2
introduce el texto y preciona enter:
holai (4717) ESP_EMISOR: Enviando a UART2: 'holai' (longitud: 4)
I (4717) ESP_EMISOR: Bytes enviados: 4
I (4717) ESP_EMISOR: Transmisión completada
>
```

On the right side of the interface, there are several tabs labeled "ESP-IDF Mo..." which appear to be monitoring or log windows.

Se recibe en esp32-2:

```
I (8958) ESP_RECEPTOR: ¡DATOS RECIBIDOS! Bytes: 4, Contenido: 'hola'
I (8958) ESP_RECEPTOR: Datos válidos, generando banner...
#  #  ###  #      ###
#  #  #  #  #  #  #
#####  #  #  #  #####
#  #  #  #  #  #  #
#  #  ###  #####  #  #
```

Circuito físico:



Conclusiones

Lerma:

La comunicación UART entre dos ESP32 permite establecer un canal confiable y eficiente para el envío y recepción de datos, siempre que se implementen mecanismos adecuados de validación y sincronización de tareas. Esto demuestra la importancia de configurar correctamente parámetros como baud rate, buffers y control de errores para garantizar una transmisión estable.

Garcia:

El despliegue de la información en la terminal a través de UART evidencia la versatilidad del ESP32 en aplicaciones embebidas, al integrar tanto la captura de datos del usuario como su procesamiento y visualización en tiempo real. Esta práctica nos da a entender el valor del UART como una herramienta fundamental en el desarrollo de sistemas de comunicación en sistemas embebidos

Dificultades en el desarrollo

Las dificultades mas notorias que experimentamos en el desarrollo, primero fue familiarizarnos con el uso del UART en el esp32, ya que cuenta con una configuración y funciones extensas, lo cual puede ser algo confuso.

Además, la toma de caracteres desde la terminal y que sea recibida por el UART del esp32, fue algo confuso de manejar, ya que se debe tener a consideración los saltos de línea y los caracteres nulos para recibir los datos correctamente.

Referencias

<https://docs.espressif.com/projects/esp-idf/en/stable/esp32/index.html>

<https://www.realdigital.org/doc/7d93df01f5ae9bcb7a0e6859badee09c>